DATA STRUCTURES
& ALGORITHMS

Complete Study Notes with Diagrams & Examples

Based on Handwritten Notes • All Topics Covered

23	12	8	30+
Sections	Algorithms	Data Structures	Diagrams

SECTION 1
DATA STRUCTURES — OVERVIEW

Definition
A Data Structure is a way of organizing and storing data in memory so it can be accessed and modified efficiently. It defines the relationship between data elements and operations.

Simple DS

- Array
- Structure / Record
- Fixed size
- Static allocation

Composite DS — Linear

- Stack (LIFO)
- Queue (FIFO)
- Linked List
- Sequential access

Composite — Non-Linear

- Tree (hierarchical)
- Graph (network)
- Multiple connections
- Dynamic structure

DSA Operations

1. Insertion
2. Deletion
3. Searching
4. Sorting
5. Updation
6. Traversal

Memory Management
4th Gen computers use Stack for data storage. Linked list uses Heap/Buffer memory. Stack = static, fast. Heap = dynamic, flexible.

SECTION 2
COMPLEXITY & ASYMPTOTIC NOTATION

Asymptotic Notations

- Big O — O(n): Upper bound, Worst case
- Omega — Ω(n): Lower bound, Best case
- Theta — Θ(n): Tight bound, Average case
- Little o — strictly smaller than upper
- Little ω — strictly larger than lower

Time-Space Trade-off

- More Space → Less Time (Faster)
- Less Space → More Time (Slower)
- Best Case: ideal input
- Average Case: typical input
- Worst Case: hardest input

Master Complexity Table

Algorithm	Best	Average	Worst	Space	Stable	Adaptive
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No	No
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Yes
Merge Sort	O(n logn)	O(n logn)	O(n logn)	O(n)	Yes	No
Quick Sort	O(n logn)	O(n logn)	$O(n^2)$	O(logn)	No	No
Linear Search	O(1)	O(n)	O(n)	O(1)	N/A	N/A
Binary Search	O(1)	O(logn)	O(logn)	O(1)	N/A	N/A
Prim's MST	O(E logV)	O(E logV)	O(E logV)	O(V)	N/A	N/A
Kruskal's MST	O(E logE)	O(E logE)	O(E logE)	O(E)	N/A	N/A
Dijkstra's	$O(V^2)$	O(V+E logV)	$O(V^2)$	O(V)	N/A	N/A

SECTION 3
STACK

Definition
LIFO — Last In First Out. Insertion and deletion from one end only (TOP). 4th Gen computers use Stack for data storage. Implemented using array or linked list.

Operations

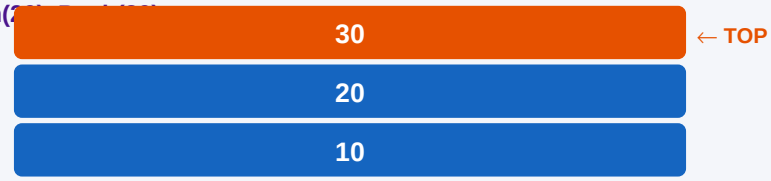
- Push — Insert at top
- Pop — Remove from top
- Peek — View top element
- isEmpty — check if empty
- isFull — check if full

Applications

- Undo / Redo in editors
- Recursion (function calls)
- Tower of Hanoi
- Expression evaluation
- String Reverse
- Tree & Graph Traversal

Stack Diagram — Example: Push 10, 20, 30 then Pop

Stack after Push(10), Push(



Step-by-Step
Push 10 → Stack:[10] Top=0 | Push 20 → Stack:[10,20] Top=1 | Push 30 → Stack:[10,20,30] Top=2 | Pop → removes 30, Top=1 | Peek → returns 20 (top element)

SECTION 4
QUEUE

Definition
Abstract Data Structure. FIFO — First In First Out. Rear End = Insertion. Front End = Deletion. If n elements, max index = (n-1). Used for CPU scheduling, print spooling.

Queue Diagram — Enqueue: 10, 20, 30, 40

Linear Queue



5 Types of Queue

Type	Description	Key Rule
1. Input Restricted	Deletion from BOTH sides	Insertion from ONE side only
2. Output Restricted	Insertion from BOTH sides	Deletion from ONE (front) side only
3. Double-Ended (Deque)	Both Insert AND Delete	From BOTH sides
4. Priority Queue	Each element has priority	Served by priority, not arrival order
5. Circular Queue	Front & Rear connected	Ring Buffer — solves wasted space

Priority Queue (values = priority)



SECTION 5
LINKED LIST

Definition
Linear data structure. Elements in sequence. Each node = [Data + Next Pointer]. Next pointer = address of next node. Last node next = NULL. Size is dynamic. Memory allocation using Heap. Easy to insert/delete.

1. Singly Linked List — Forward direction only

Singly Linked List: [data|next→] ... NULL



2. Doubly Linked List — Forward & Backward

Doubly Linked List: [←prev|data|next→]



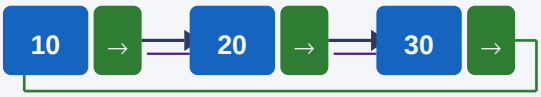
3. Circular Linked List — Last points to First

Circular Singly: Last node → First node



4. Doubly Circular — Both directions + Circular

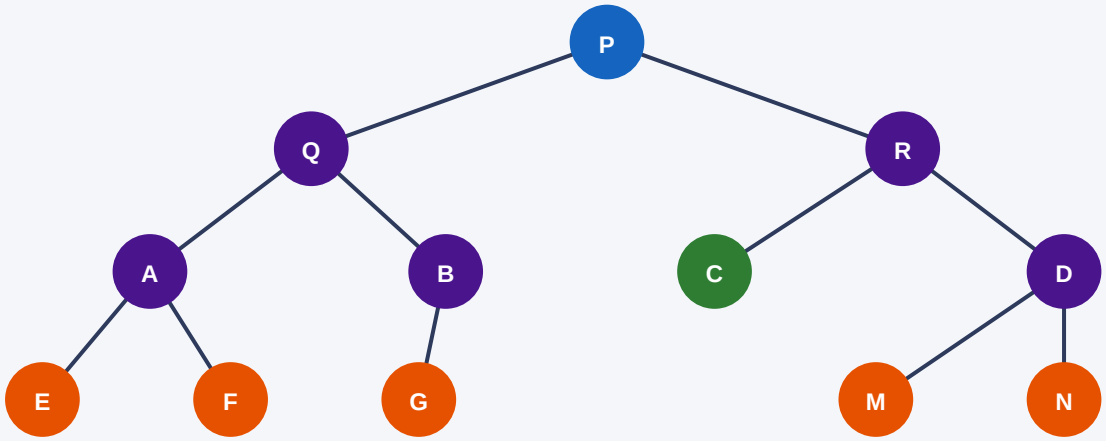
Doubly Circular: bidirectional + circular



SECTION 6
TREE — CONCEPTS & TYPES

Definition
Non-linear, hierarchical data structure. Root = topmost node. Node = container. Key = value. Internal node has at least one child. Leaf node has no children. Degree = total child nodes.

General Tree — Root(P), Internal Nodes(Q,R,A,B,C,D), Leaf Nodes(E,F,G,M,N)



Tree Terminology

Term	Definition	Example from tree above
Root	Topmost node, no parent	P
Internal Node	Has at least one child	Q, R, A, B, C, D
Leaf Node	No children	E, F, G, M, N (orange)
Degree	Total number of children	Degree of P=2, A=2, D=2
Height	Longest path from node to leaf	Height of P = 3
Depth	Distance from root to node	Depth of E = 3
Sibling	Nodes with same parent	Q & R are siblings
Subtree	Node + all its descendants	Subtree at Q = {Q,A,B,E,F,G}

SECTION 6b
TREE TYPES & TRAVERSAL

Types of Tree

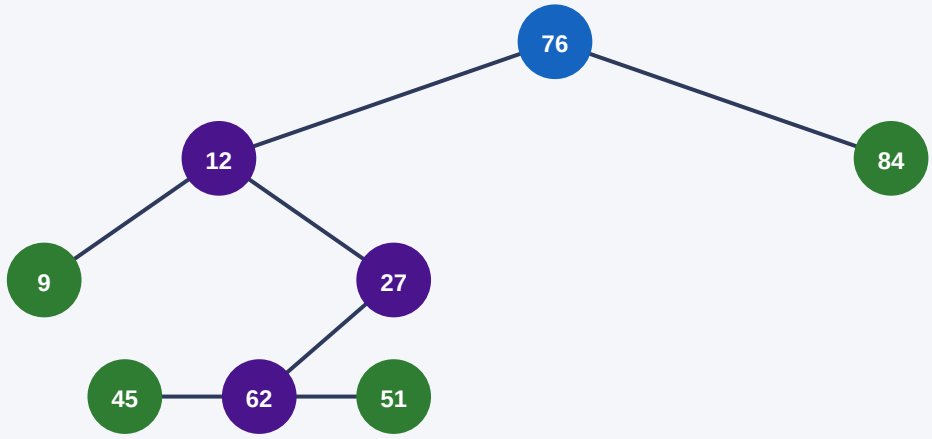
- 1. General Tree — no constraint, infinite children
- 2. Binary Tree — max 2 children per node
- 3. BST — left<root<right, searchable
- 4. AVL Tree — self-balancing, BF={-1,0,+1}
- 5. Red-Black — level-wise coloring, balanced

BST Rule

- Left child < Parent < Right child
- Searching easy — O(log n)
- Balanced tree
- Nodes sequentially arranged
- BF = Height(Left) - Height(Right)

BST Example (from notes): 76,12,84,9,27,62,45,51 — First = Root

Binary Search Tree: left<parent<right



Tree Traversal Methods

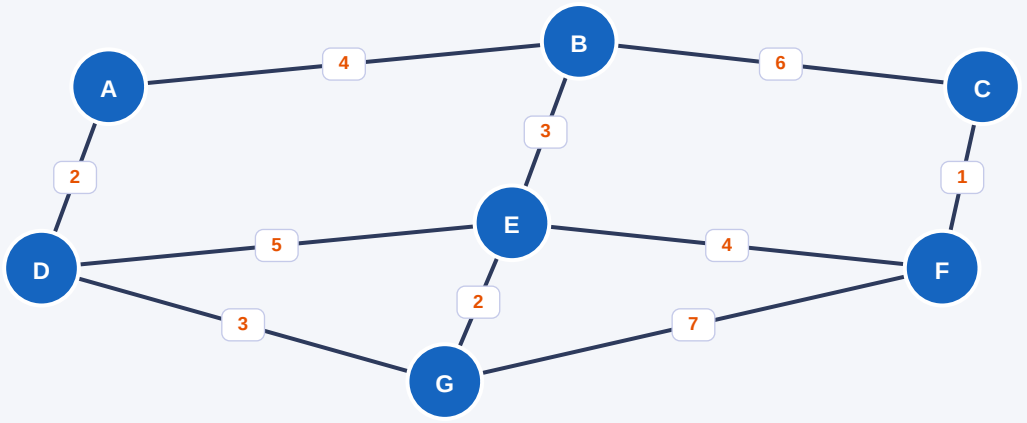
Method	Order	Mnemonic	Result on BST above
In-Order	Left → Root → Right	LRoR	Gives SORTED output
Pre-Order	Root → Left → Right	RoLR	Copy tree structure
Post-Order	Left → Right → Root	LRRo	Delete tree safely

SECTION 7
GRAPH — TYPES & REPRESENTATION

Definition
Collection of Nodes (Vertices) and Edges. G=(V,E). V(G)=set of vertices. E(G)=set of edges. Connected path called a node. G = (V, E).

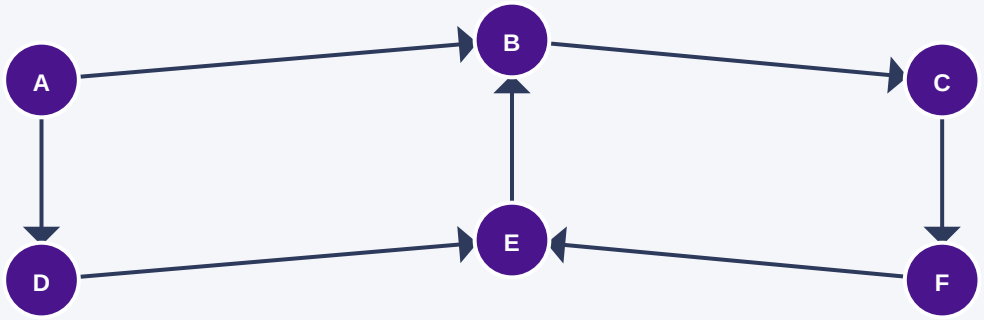
Undirected Weighted Graph — Example used for MST algorithms

Undirected Weighted Graph G(7 nodes, 10 edges)



Directed Graph (Digraph) — edges have one direction only

Directed Graph — arrows show direction



9 Types of Graphs (from notes)

Type	Description	Key Property
1. Undirected	No direction on edges	Bidirectional — {A,B}={B,A}
2. Directed (Digraph)	Edges have direction	{A,B} ≠ {B,A}
3. DAG	Directed + No cycles	Directed Acyclic Graph
4. Multi-Graph	Multiple edges between nodes	Loops allowed
5. Simple Graph	No parallel edges	No loops allowed
6. Weighted Graph	Edges have weight/cost	Used for shortest path
7. Complete Graph	Every node connected	Fully connected mesh
8. Connected Graph	Every node has ≥1 edge	No isolated vertices
9. Null Graph	Zero edges	No connectivity

SECTION 7b
GRAPH TERMINOLOGY & REPRESENTATION

Graph Terms

- Walk: move from node to node
- Trail: edges not repeated
- Circuit: closed trail
- Cycle: closed trail (= Circuit)
- Loop: edge from node to itself
- Adjacent: nodes sharing edge

Degree Types

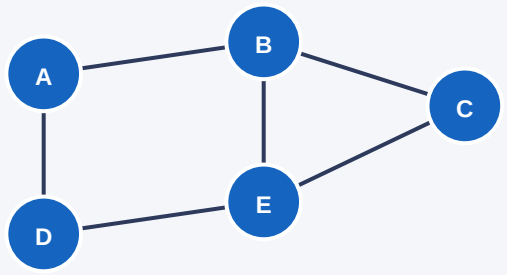
- Degree: total connected edges
- In-degree: inward edges (directed)
- Out-degree: outward edges (directed)
- Isolated: degree = 0
- Universal: connected to all others

Graph Representation — Adjacency Matrix

Adjacency Matrix Rule

Rule 1: If node has neighbour → degree=1. Rule 2: No neighbour → degree=0. For weighted graph, use weight value. Undirected: 2N nodes in adj list. Directed: N nodes.

Graph G



	A	B	C	D	E
A	0	1	0	1	0
B	1	0	1	0	1
C	0	1	0	0	1
D	1	0	0	0	1
E	0	1	1	1	0

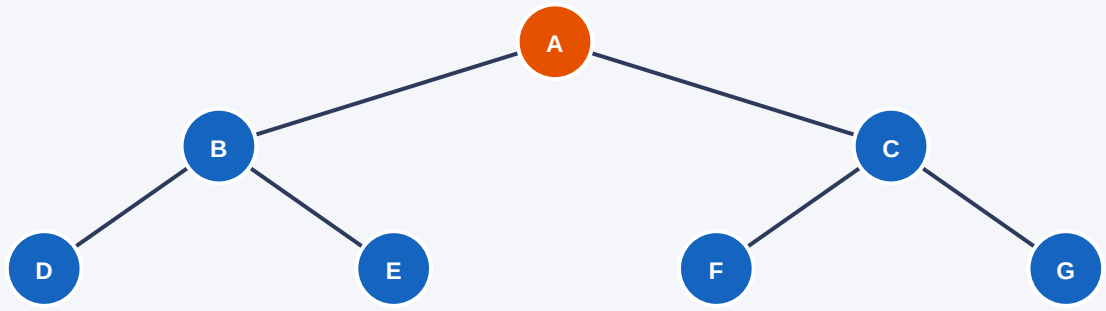
Adjacency List

Adj(A)=B,D | Adj(B)=A,C,E | Adj(C)=B,E | Adj(D)=A,E | Adj(E)=B,C,D. Uses Linked List. Only checks connectivity. Space-efficient for sparse graphs.

SECTION 8
BFS — BREADTH FIRST SEARCH

Definition
Breadth First Search. Traversal = level by level (A→B→C→D→E→F→G). Uses Queue (FIFO). Slower than DFS. Less memory. Best if target is near root. First sibling then child. Back-tracking NOT allowed.

BFS Tree — Traversal: A→B→C→D→E→F→G

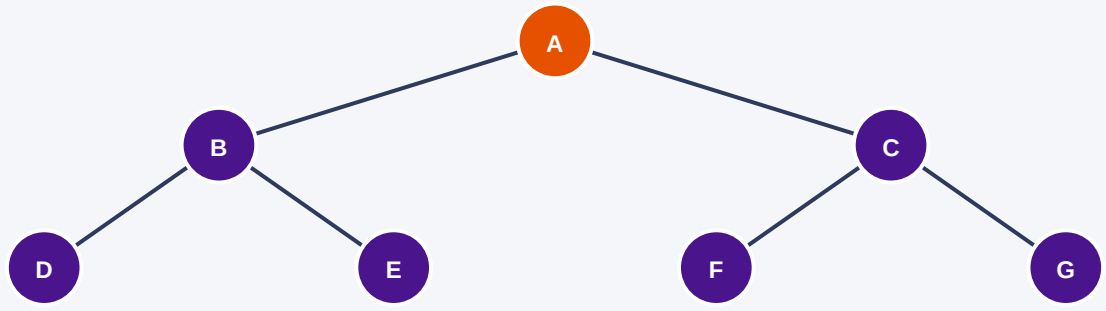


BFS Steps
1. Start at A (root), mark visited, add to Queue: [A] | 2. Dequeue A, visit neighbors B,C → Queue:[B,C] | 3. Dequeue B, visit D,E → Queue:[C,D,E] | 4. Dequeue C, visit F,G → Queue:[D,E,F,G] | 5. Dequeue D,E,F,G (no new neighbors) → Done. Final order: A B C D E F G

SECTION 9
DFS — DEPTH FIRST SEARCH

Definition
Depth First Search. Goes deep first (A→B→D→E→C→F→G). Uses Stack (LIFO). Faster. More memory. Back-tracking allowed. First child then sibling.

DFS Tree — Traversal: A→B→D→E→C→F→G



DFS Steps
1. Start at A, push to Stack: [A] | 2. Pop A, visit → push neighbors: Stack:[C,B] | 3. Pop B → push D,E: Stack:[C,E,D] | 4. Pop D (no neighbors) → Stack:[C,E] | 5. Pop E → Stack:[C] | Pop C → push F,G | Pop G,F | Done. Final order: A B D E C G F

BFS vs DFS Comparison

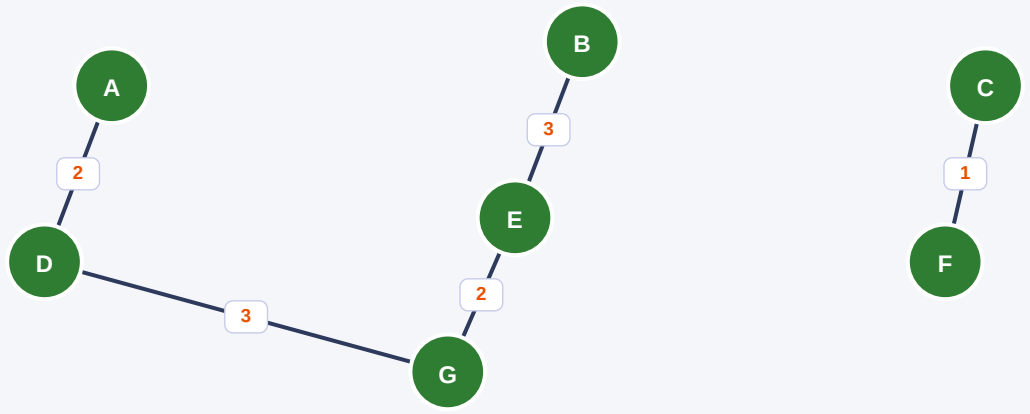
Property	BFS	DFS
Data Structure	Queue (FIFO)	Stack (LIFO)
Order	Level by level	Depth first
Memory	Less	More
Back-tracking	Not allowed	Allowed
Speed	Slower	Faster
Best for	Target near root	Deep paths, cycles
First Access	Sibling → Child	Child → Sibling
Shortest Path	Yes (unweighted)	Not guaranteed

SECTION 9
SPANNING TREE & MST ALGORITHMS

Spanning Tree
Subgraph of undirected graph including ALL nodes with MINIMUM edges to keep it connected. V vertices → exactly V-1 edges. No cycles. Coined by Radia Perlman. Possible spanning trees for N nodes = N^(N-2). Cost = sum of all edge weights.

MST Example on Weighted Graph

Minimum Spanning Tree (MST) — Cost: 2+2+1+3+3+4=selected edges



SECTION 10
PRIM'S ALGORITHM [O(E log V)]

Prim's Definition
Vertex-centric greedy algorithm. Start from any arbitrary vertex. Examine edges to adjacent unvisited vertices. Connect vertex with minimum cost edge. Repeat until all vertices connected. Do NOT form cycles. First Node, then Edge.

Step	Action	MST Cost So Far
1	Remove all cycles, loops, parallel edges	0
2	Choose arbitrary root (e.g. A), add to MST	0
3	Find cheapest edge from visited set	min edge weight
4	Add destination vertex + edge to MST	cumulative
5	Repeat steps 3-4 for all vertices	Total MST cost
Done	All V vertices connected with V-1 edges	Minimum total

SECTION 11
KRUSKAL'S ALGORITHM [O(E log E)]

Kruskal's Definition
Edge-centric greedy algorithm. Sort all edges by weight. Pick cheapest edge that does NOT create a cycle. Use Union-Find to detect cycles. Continue until V-1 edges added. Do NOT create loops/cycles. Edge first, Node last. Traverse each vertex once.

Kruskal's Walkthrough on Graph (7 nodes)

Edge	Weight	Action	MST?
C-F	1	No cycle formed	✓ Add
A-D	2	No cycle formed	✓ Add
E-G	2	No cycle formed	✓ Add
B-E	3	No cycle formed	✓ Add
D-G	3	No cycle formed	✓ Add
E-F	4	No cycle formed	✓ Add
A-B	4	Would form cycle!	✗ Skip
D-E	5	Would form cycle!	✗ Skip
B-C	6	No cycle formed	✓ Add (V-1=6 edges)

Prim's vs Kruskal's

- Vertex-centric (grow from 1 vertex)
- Uses Priority Queue (min-heap)
- Complexity: O(E log V)
- Best for dense graphs (E ≈ V²)
- No specific start, can start anywhere

Kruskal's

- Edge-centric (sort all edges first)
- Uses Union-Find structure
- Complexity: O(E log E)
- Best for sparse graphs
- Edge first, Node last approach

SECTION 12
SORTING — CONCEPTS

3 Sorting Patterns

- A. Inplace vs Non-Inplace
- B. Stable vs Unstable
- C. Adaptive vs Non-Adaptive

Pattern A — Memory

- Inplace: no extra memory (Bubble, Selection, Insertion, Quick)
- Non-Inplace: needs O(n) extra space (Merge Sort)

Pattern B — Stability

- Stable: equal elements keep original order
- Stable sorts: Merge, Insertion, Bubble

Pattern C — Adaptivity

- Adaptive: benefits from pre-sorted input
- Adaptive: Bubble, Insertion Sort
- Non-Adaptive: order still changed (Selection, Quick)

SECTION 13
BUBBLE SORT

Definition

Compare each adjacent pair; swap if out of order. Each pass bubbles largest element to end. Pass terminates early if no swap. Inplace, Stable, Adaptive. Best O(n), Worst O(n²).

Bubble Sort Example: [5,1,4,2,8]

Pass	Comparisons & Swaps	Result After Pass
Pass 1	5>1 swap→[1,5,4,2,8] | 5>4 swap→[1,4,5,2,8] | 5>2 swap→[1,4,2,5,8] | 5<8 no swap	[1,4,2,5,8] → 8 fixed
Pass 2	4>2 swap→[1,2,4,5,8] | 4<5 no swap | 5<8 no swap	[1,2,4,5,8] → 5 fixed
Pass 3	No swaps → EARLY EXIT (already sorted!)	[1,2,4,5,8] ✓ Done

SECTION 14
SELECTION SORT

Definition

Find minimum in unsorted part, swap to current position. Repeat. Always O(n²). Makes exactly n-1 swaps. Inplace, Unstable, Non-Adaptive.

Pass	Finds Min	Swap	Array State
Pass 1	min=12 at idx 3	swap[0]↔[3]	[12, 16, 84, 42, 77, 26, 53]
Pass 2	min=16 at idx 1	no swap	[12, 16, 84, 42, 77, 26, 53]
Pass 3	min=26 at idx 5	swap[2]↔[5]	[12, 16, 26, 42, 77, 84, 53]
Pass 4	min=42 at idx 3	no swap	[12, 16, 26, 42, 77, 84, 53]
Pass 5	min=53 at idx 6	swap[4]↔[6]	[12, 16, 26, 42, 53, 84, 77]
Pass 6	min=77 at idx 6	swap[5]↔[6]	[12, 16, 26, 42, 53, 77, 84] ✓

SECTION 15
INSERTION SORT

Definition
Maintains growing sorted sublist. For each element, scan left to find correct position and insert. Starts from index 1. Best O(n) when sorted, O(n²) average/worst. Inplace, Stable, Adaptive.

Step	Key	Action	Array State
1	12	12<85, shift 85 right, insert 12	[12,85,59,45,72,51]
2	59	59<85, shift, insert	[12,59,85,45,72,51]
3	45	45<85,45<59, shift both, insert	[12,45,59,85,72,51]
4	72	72<85, shift, insert	[12,45,59,72,85,51]
5	51	51<85,72,59, shifts, insert	[12,45,51,59,72,85] ✓

SECTION 16
MERGE SORT

Definition
Divide and Conquer. Most efficient algorithm. O(n log n) for all cases. Stable. NOT inplace (O(n) extra space). Split → Sort → Merge.

Merge Sort: [5,2,4,7,1,3,2,6]
DIVIDE: [5,2,4,7]|[1,3,2,6] → [5,2]|[4,7]|[1,3]|[2,6] → [5]|[2]|[4]|[7]|[1]|[3]|[2]|[6] | MERGE: [2,5]|[4,7]|[1,3]|[2,6] → [2,4,5,7]|[1,2,3,6] → FINAL: [1,2,2,3,4,5,6,7] ✓

SECTION 17
QUICK SORT

Definition
Divide and Conquer. Choose pivot (rightmost element). Partition: elements<pivot go left, >pivot go right. Pivot is in final position. Recurse on left and right. Best/Average O(n log n), Worst O(n²). Inplace. NOT Stable, NOT Adaptive.

Quick Sort: [10,15,1,2,6,12,5,7] → Pivot=7
Partition: elements<7=[1,2,6,5], pivot=7, elements>7=[10,15,12] | After partition: [1,2,6,5, 7, 10,15,12] | Left subarray [1,2,6,5] → pivot=5: [1,2,5,6] | Right [10,15,12] → pivot=12: [10,12,15] | FINAL: [1,2,5,6,7,10,12,15] ✓

SECTION 18
SEARCHING — LINEAR & BINARY

Linear Search

- Search sequentially until found
- Works on SORTED or UNSORTED data
- Complexity: O(n)
- Best case: O(1) — first element
- Space: O(1)
- Simple but slow for large data

Binary Search

- Works ONLY on SORTED array
- Compare with middle element
- If target<mid → search left half
- If target>mid → search right half
- Complexity: O(log n)
- Best for large sorted datasets

Binary Search Example: find 42 in [3,8,15,22,31,42,57,68,74,85,91]

Step 1: Low=0, High=10, Mid=5 → arr[5]=42 → arr[5]<42? No. arr[5]=42 → FOUND at index 5! (Best case: found at first mid!). If not found example — search 50: Step1: mid=5=42<50→search right. Step2: mid=8=74>50→search left. Step3: mid=6=57>50→search left. Step4: mid=5→already checked. Not found.

SECTION 19
ARITHMETIC EXPRESSION EVALUATION

Definition
Combination of arithmetic operators & operands/variables. Operators solved based on Precedence: ^ (highest) → /,* → +,- (lowest). Brackets solved first.

Three Notations

- Infix: A + B (operator between)
- Prefix/Polish: + A B (operator before)
- Postfix/RPN: A B + (operator after)

Precedence Rules

- Highest: ^ (Power/Exponent)
- Next: / and * (left to right)
- Lowest: + and - (left to right)
- Brackets: always solved first

Infix Expression	Prefix (Polish)	Postfix (RPN)
(A+B)*C	*+ABC	AB+C*
A+B*C-D	+-A*BCD	ABC*+D-
5*(6+2)-12/4	-*5+62/124	562+*124/-
A+(B*C-(D/E^F)*G)*H	+A*-*BC*/DEFGH	ABC*DEF^/G*-H*+

SECTION 20
SHORTEST PATH & ADVANCED ALGORITHMS

Floyd-Warshall

- All Pairs Shortest Path (APSP)
- Works with negative weights
- Time: $O(V^3)$
- Dynamic programming approach

Dijkstra's Algorithm

- Single Source Shortest Path
- No negative weights allowed
- $O(V^2)$ with adjacency matrix
- $O(V+E \log V)$ with adjacency list

Multi-Stage Graph

A graph whose vertices are divided into multiple levels. Edges go only from one stage to the next stage. Example: Shortest-path problem. Note: All trees are graphs, but not all graphs are trees.

SECTION 21
CLASSICAL ALGORITHMIC PROBLEMS

8-Queen Problem

- Place 8 queens on 8×8 chessboard
- No two queens attack each other
- No sharing: row, column, diagonal
- Approach: BACKTRACKING
- Total solutions: 92

Hamiltonian Problem

- From graph theory
- Path visiting every node exactly once
- Hamiltonian PATH: visits all vertices once
- Hamiltonian CYCLE: path + return to start
- Approach: Backtracking

SECTION 22
NP — NON-DETERMINISTIC POLYNOMIAL

NP Definition

Problems whose SOLUTION can be verified in polynomial time. Finding the solution may take exponential time.

Category	Examples	Description
NP	Most graph problems	Verifiable in polynomial time
NP-Hard	Traveling Salesman (TSP), 3D Matching	At least as hard as NP problems
NP-Complete	SAT, Hamiltonian Cycle, Clique, Vertex Cover, 0/1 Knapsack	In both NP and NP-Hard

SECTION REF
QUICK REFERENCE — COMPLETE COMPLEXITY CHART

Algorithm	Best	Average	Worst	Space	Stable	In-place
Bubble Sort	O(n)	O(n²)	O(n²)	O(1)	Yes	Yes
Selection Sort	O(n²)	O(n²)	O(n²)	O(1)	No	Yes
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)	Yes	Yes
Merge Sort	O(n logn)	O(n logn)	O(n logn)	O(n)	Yes	No
Quick Sort	O(n logn)	O(n logn)	O(n²)	O(logn)	No	Yes
Linear Search	O(1)	O(n)	O(n)	O(1)	-	-
Binary Search	O(1)	O(logn)	O(logn)	O(1)	-	-
BFS	O(1)	O(V+E)	O(V+E)	O(V)	-	-
DFS	O(1)	O(V+E)	O(V+E)	O(V)	-	-
Prim's	O(E logV)	O(E logV)	O(E logV)	O(V)	-	-
Kruskal's	O(E logE)	O(E logE)	O(E logE)	O(E)	-	-
Dijkstra's	O(V²)	O(V+E logV)	O(V²)	O(V)	-	-
Floyd-Warshall	O(V³)	O(V³)	O(V³)	O(V²)	-	-

DS Quick Reference

Data Structure	Type	Access	Insert	Delete	Space
Array	Linear	O(1)	O(n)	O(n)	O(n)
Stack	Linear	O(n)	O(1)	O(1)	O(n)
Queue	Linear	O(n)	O(1)	O(1)	O(n)
Singly Linked List	Linear	O(n)	O(1)	O(1)	O(n)
Doubly Linked List	Linear	O(n)	O(1)	O(1)	O(n)
Binary Search Tree	Non-Linear	O(logn)	O(logn)	O(logn)	O(n)
AVL Tree	Non-Linear	O(logn)	O(logn)	O(logn)	O(n)
Graph (Adj Matrix)	Non-Linear	O(1)	O(V²)	O(V²)	O(V²)
Graph (Adj List)	Non-Linear	O(V)	O(1)	O(E)	O(V+E)